

Hospital Management System



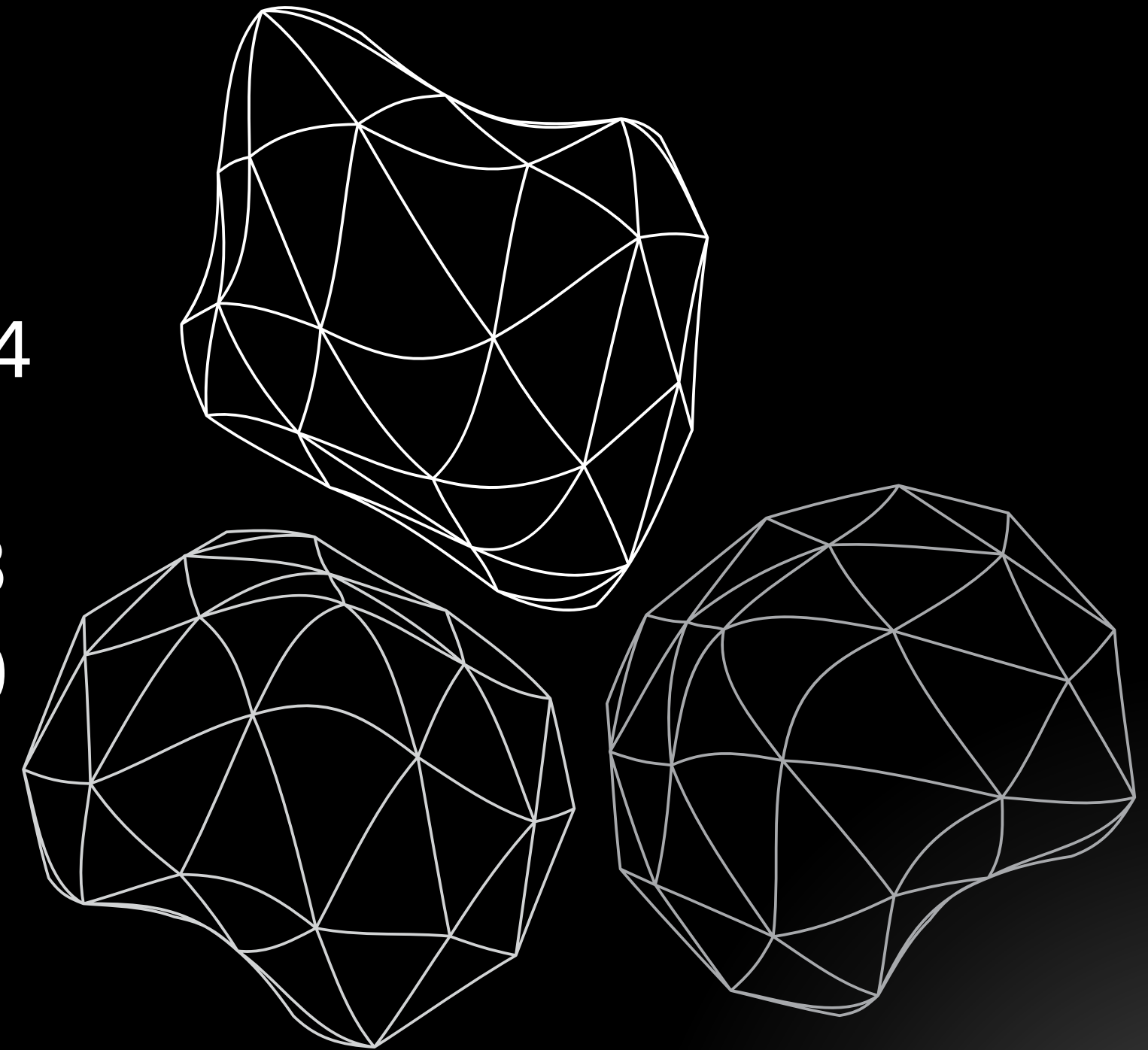
PRESENTED BY:

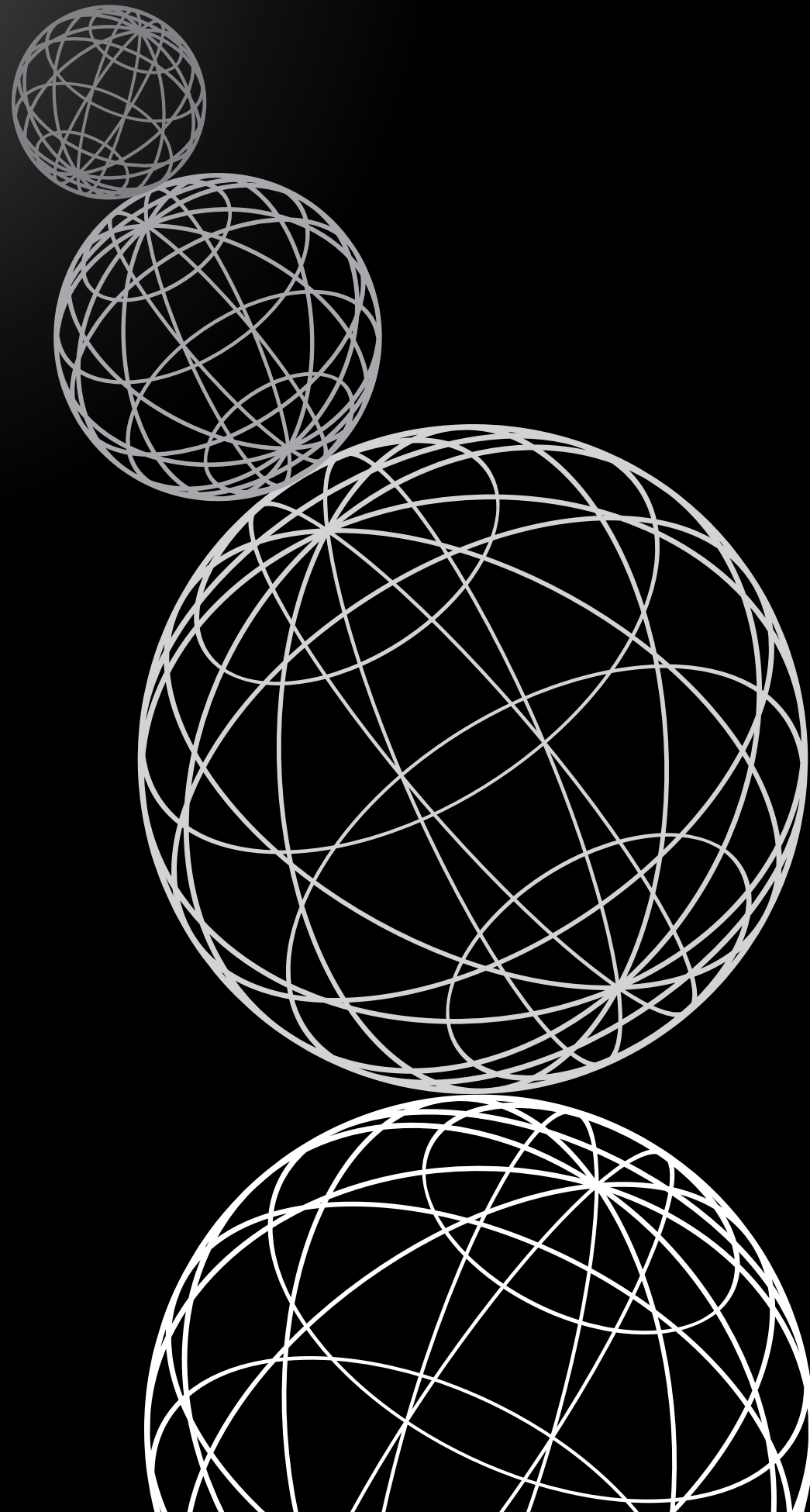
Hasini Devarakonda | AP24110011274

Tanvi Vallamkonda | AP24110011313

E.D.S Harshitha | AP24110011178

Leena | AP24110011180



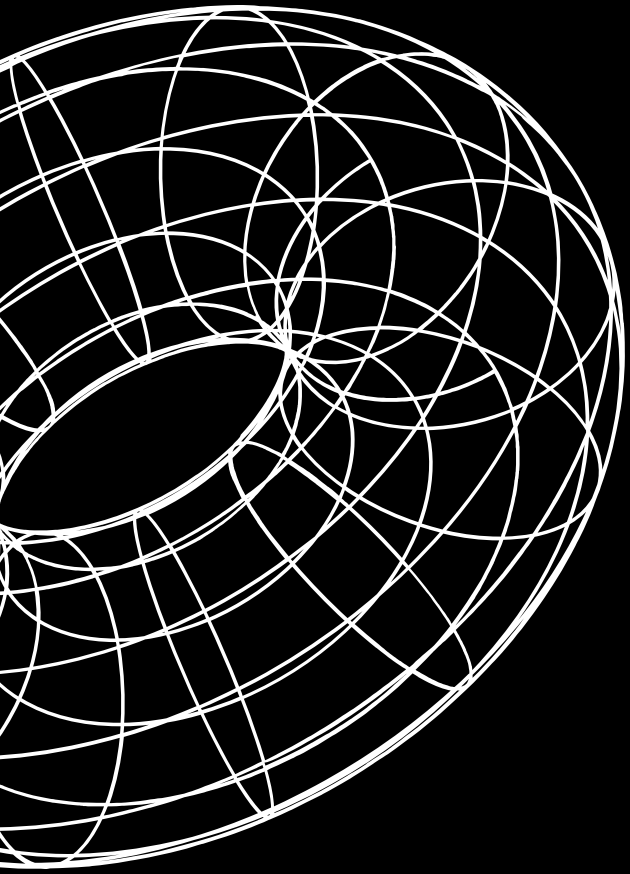


INTRODUCTION

- A Hospital Management System is a database system used to store and manage hospital information.
- It keeps records of patients, doctors, appointments, treatments, and staff.
- The system helps in efficient data storage, easy retrieval, and reduced manual errors.
- Implemented using MySQL database.

OBJECTIVE

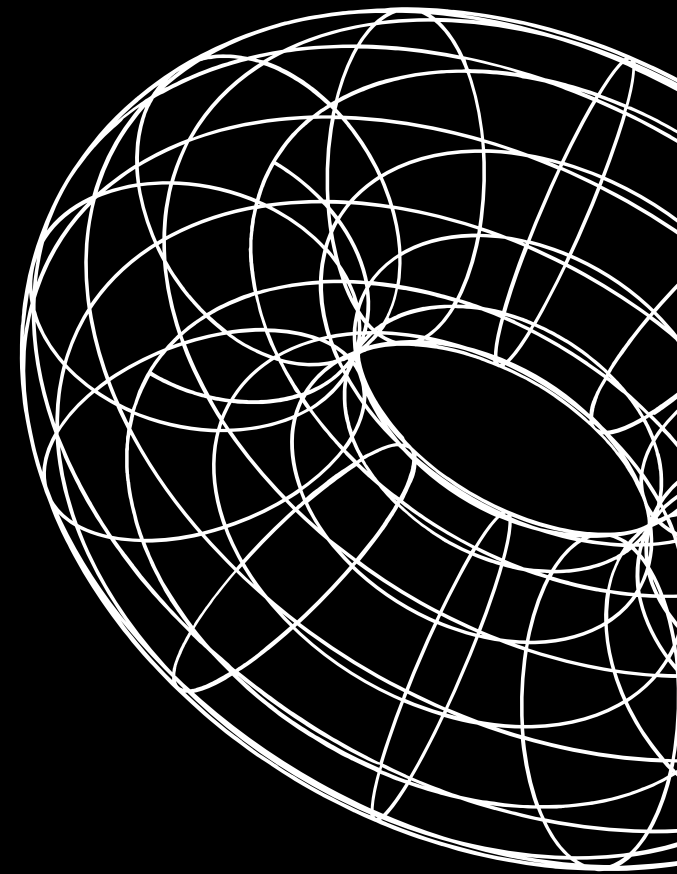
- To design a structured hospital database.
- To understand Relational Database concepts.
- To implement Primary Keys and Foreign Keys.
- To perform CRUD operations using SQL queries.



TABLES USED IN THE PROJECT

1. Patient
2. Doctor
3. Appointment
4. Billing
5. staff

Each table stores specific hospital information.





Patient table

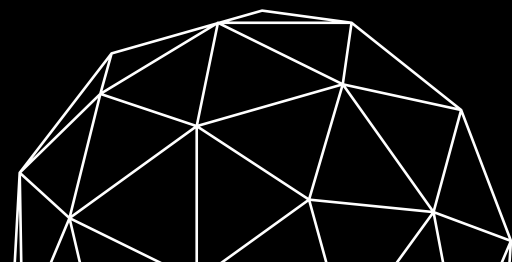
```
CREATE TABLE patient (  
  p_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  age INT,  
  gender VARCHAR(20),  
  phone_num VARCHAR(15),  
  address VARCHAR(100)  
  );
```

Field	Type	Null	Key	Default	Extra
p_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
age	int	YES		NULL	
gender	varchar(20)	YES		NULL	
phone_num	varchar(15)	YES		NULL	
address	varchar(100)	YES		NULL	

Keys:

Primary Key → p_id

Purpose: Stores patient details.

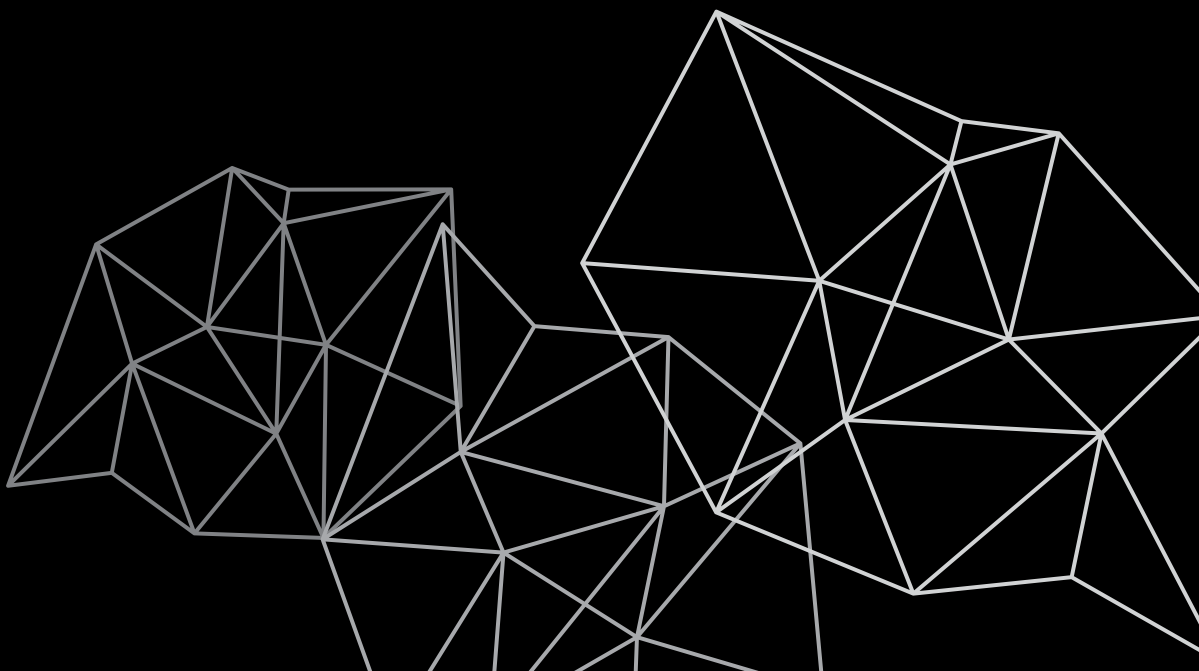


Doctor table

```
CREATE TABLE doctor (  
  d_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  specialization VARCHAR(100),  
  phone VARCHAR(15)  
  );
```

Field	Type	Null	Key	Default	Extra
d_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
specialization	varchar(100)	YES		NULL	
phone	varchar(15)	YES		NULL	

Keys:
Primary Key → d_id
Purpose: Stores doctor details.



Appointment table

```
CREATE TABLE appointment (  
  appointment_id INT PRIMARY KEY,  
  p_id INT,  
  d_id INT,  
  date DATE,  
  time TIME,  
  FOREIGN KEY (p_id) REFERENCES  
    patient(p_id),  
  FOREIGN KEY (d_id) REFERENCES  
    doctor(d_id)  
);
```

Field	Type	Null	Key	Default	Extra
appointment_id	int	NO	PRI	NULL	
p_id	int	YES	MUL	NULL	
d_id	int	YES	MUL	NULL	
date	date	YES		NULL	

Keys:

Primary Key → appointment_id

Foreign Key → p_id references patient

Foreign Key → d_id references doctor

Purpose: Connects patients and doctors.

Billing table

```
CREATE TABLE billing (  
  bill_id INT PRIMARY KEY,  
  p_id INT,  
  amount DECIMAL(10,2),  
  payment_status VARCHAR(20),  
  FOREIGN KEY (p_id) REFERENCES patient(p_id)  
  );
```

Field	Type	Null	Key	Default	Extra
bill_id	int	NO	PRI	NULL	
p_id	int	YES	MUL	NULL	
amount	decimal(10,2)	YES		NULL	
payment_status	varchar(20)	YES		NULL	

Keys:

Primary Key → bill_id

Foreign Key → p_id

Purpose: Stores patient billing details.

Staff table

```
CREATE TABLE staff (  
  staff_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  role VARCHAR(50),  
  phone VARCHAR(15),  
  salary DECIMAL(10,2)  
);
```

```
mysql> create table staff( staff_id int primary key, name varchar(100), role varchar(50), phone varchar(15), salary decimal(10,2));  
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> describe staff;
```

Field	Type	Null	Key	Default	Extra
staff_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
role	varchar(50)	YES		NULL	
phone	varchar(15)	YES		NULL	
salary	decimal(10,2)	YES		NULL	

```
5 rows in set (0.05 sec)
```

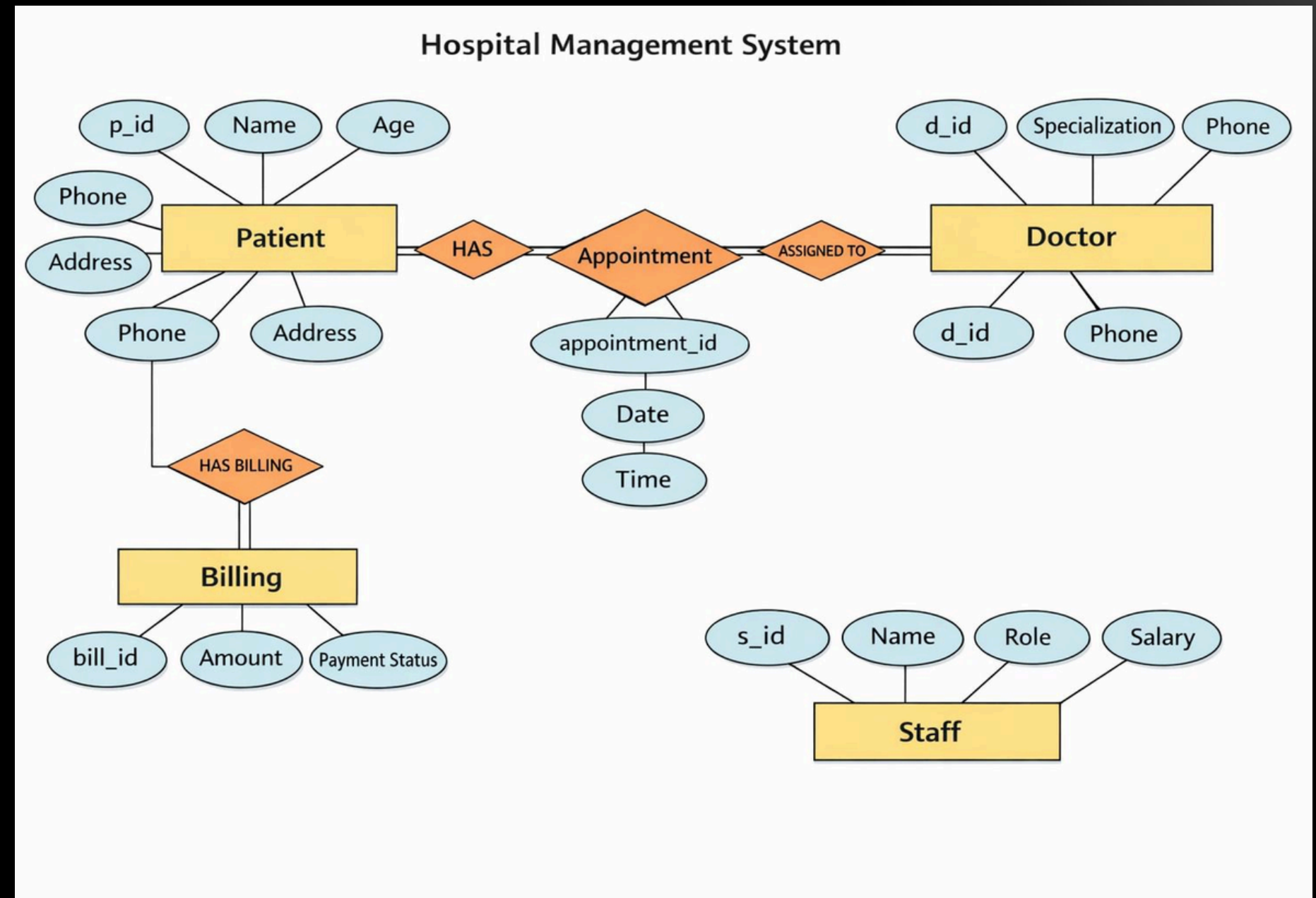
Keys:

Primary Key → staff_id

Purpose: Stores hospital employee details.

Table Relationships

- Patient and Doctor are connected through Appointment table
- Appointment table contains foreign keys
- Billing table stores patient payments
- Staff table stores hospital employees



SQL Queries

1. Display all staff records

```
SELECT * FROM staff;
```

2. Count number of staff in each role

```
SELECT role, COUNT(*)  
FROM staff  
GROUP BY role;
```

3. Calculate total salary for each role

```
SELECT role, SUM(salary) AS total_salary  
FROM staff  
GROUP BY role;
```

4. Join query to show patient appointments with doctors

```
SELECT patient.name, doctor.name, appointment.date  
FROM appointment  
JOIN patient ON appointment.p_id = patient.p_id  
JOIN doctor ON appointment.d_id = doctor.d_id;
```

Conclusion

- The Hospital Management System helps manage hospital data efficiently.
- It stores patient, doctor, appointment, staff, and billing information.
- Using DBMS improves organization and data retrieval.

The background is a dark gradient with white, wavy, wireframe-like lines that create a sense of depth and movement. These lines are concentrated in the corners and sides, framing the central text.

THANK YOU